Course: CSA0919 – Programming in Java
Assignment: Campus Smart-Parking System
Student Name: Juhi Surin
Register Number: 192525148
Java Version: Java 27

TASK 2 – Choose the Data Representation (3 Marks)

---

1. Objective

This task selects the Java data types and the array structure used to represent the data of the Campus Smart-Parking System. It also analyses where type conversion and casting are needed, how a wrong conversion produces a wrong fee, how input is read and validated, and how the chosen floor-and-slot representation compares with two alternatives in time and space complexity.

The report keeps what the assignment states separate from what is assumed here.

Table 1: Stated in the assignment and assumed in this report

Stated in the assignment	Assumed in this report
Facilities of 1 × 20, 3 × 50 and 5 × 100 slots; each floor has a fixed number of numbered slots. The slots are held in a jagged array String[floor][slot], where an element holds a vehicle number or null when the slot is free. Every started hour is charged in full; billable hours are the ceiling of minutes / 60 using integer arithmetic. Hourly rates are Rs 10 (two-wheeler) and Rs 30 (car), and EV charging is Rs 12 per kWh. Duration is an int number of minutes and fees are double values. Input is read with Scanner or command-line arguments (the choice is mine).	Floors and slots are numbered from 0 inside the program (Task 1 assumption). Entry and exit times are held as LocalDateTime values and the exit time is supplied as input rather than read from the system clock. A registration number is 4 to 12 letters or digits after trimming and conversion to upper case. A stay of zero minutes has no started hour and costs Rs 0. The capacities used in the code snippets (for example 20, 50 and 40) are illustrative.

2. Data Types for Each Field

Table 2: Java data type chosen for each field

Field	Java type	Example value	Justification
Slot number	int	7	Position of the slot within its floor, counted from 0. It is used directly as an array index, which Java requires to be an int. The largest facility needs only 0 to 99.

Field	Java type	Example value	Justification
Floor number	int	1	Index of the floor row in the slot structure, counted from 0. At most 5 floors are stated, so int is more than sufficient.
Vehicle registration number	String	"TN01AB1111"	It mixes letters and digits and its leading zeros (the “01”) are significant, so a numeric type would lose information. No arithmetic is done on it. It is the value stored in slots[floor][slot], where null means the slot is free.
Vehicle type	enum VehicleType (TWO_WHEELER, CAR, ELECTRIC_CAR)	ELECTRIC_CAR	The set of values is fixed. Unlike a String, a misspelt constant fails at compile time. It is used when input is validated to decide which Vehicle subclass to create; the behaviour itself stays in the subclasses (Task 1 design).
Entry time	java.time.LocalDateTime	2026-11-02T09:15	Holds date and time in one immutable value. The date is needed for monthly-pass validity (Task 1), and Duration.between handles a stay that crosses midnight. Invalid values are rejected when parsed. It is kept to minute precision and stored in the ParkingTicket.
Exit time	java.time.LocalDateTime	2026-11-02T10:16	Same reasons as the entry time. It is supplied as input so that the same data always gives the same fee.
Parking duration	int (whole minutes)	61	Derived from entry and exit time and matches the assumption of whole minutes. The int range (up to 2,147,483,647 minutes) is far larger than any real stay, and an integer allows exact integer arithmetic for billing.
Billable hours	int	2	A whole number of hours obtained from the duration by ceiling division (Section 5).
Hourly rate	double	30.0	Rupees per hour. The rates in the assignment are whole rupees, and double matches the ratePerHour() type used in the assignment’s fee code.
Parking fee	double	60.0	Billable hours (int) × hourly rate (double). The rates and hours are whole numbers, so the result is exact (see the note below the table). It matches the double fee type in the assignment’s fee code.

Field	Java type	Example value	Justification
Ticket number	long	1001	A counter that only increases, so it needs arithmetic (+1) and ordering, which a String does not give. long is chosen so that the counter cannot realistically overflow and break the rule that a ticket number is never reused; int (about 2.1 billion tickets) would also be adequate.
Slot occupancy status	boolean (derived, not stored)	true	The question has exactly two answers. It is obtained from the slot itself as slots[floor] [slot] != null. A separate boolean[][] is not kept because it could disagree with the String[][] and create two sources of truth.

Why double is acceptable for the fee. Every amount in this system is a whole number of hours multiplied by a whole-rupee rate (Rs 10, Rs 30, Rs 12 per kWh). A double represents every integer up to 2^53 exactly, so results such as 2 × 30.0 = 60.0 and 10 × 12.0 = 120.0 carry no rounding error, and double keeps the design consistent with the assignment's fee code. BigDecimal would be required if fees contained fractions of a rupee (paise), percentage tax or discounts, or if many amounts were totalled in an account, because decimal fractions such as 0.1 have no exact binary representation. None of these is part of the stated problem.

Limitations of the chosen types.

- int overflows beyond 2,147,483,647. This is irrelevant for floor and slot numbers but matters for duration arithmetic, which is handled in Section 5.
- The type String does not check the format of a registration number, so the format is validated in code (Section 4); the pattern itself is an assumption.
- An enum must be edited to add a new vehicle category. This is a one-line, compiler-checked change and is acceptable because new categories are rare.
- LocalDateTime carries no time zone or daylight-saving information. This is adequate for one campus using one local clock, and minute precision is enforced by the input format.
- double cannot represent most decimal fractions exactly, so it is safe only while prices stay in whole rupees.
- The long ticket counter is held in memory only, so it restarts when the program restarts (data is kept in memory only, as assumed in Task 1).

Why the types are suitable. They are the simplest types that hold the data without loss, they match the types used in the assignment's own code, they are large enough for the facility sizes of up to 5 floors × 100 slots, and they allow the validation and exact arithmetic that the later billing relies on.

3. Representing Floors and Parking Slots

Table 3: One-dimensional, two-dimensional and jagged arrays

Aspect	One-dimensional array	Two-dimensional (rectangular) array	Jagged array
Declaration	String[] slots = new String[floors * slotsPerFloor];	String[][] slots = new String[floors] [slotsPerFloor];	String[][] slots = new String[floors][]; then slots[f] = new String[capacity[f]];
How floors and slots are represented	One block; the slot is at index = floor × slotsPerFloor + slot.	A table; row = floor, column = slot.	An array of rows; row f is a separate array holding the slots of floor f.
Different numbers of slots on different floors	No. The formula assumes the same slotsPerFloor for every floor.	No. Every row has the same length, so shorter floors need padding cells.	Yes. Each row has its own length, available as slots[f].length.
Behaviour with an invalid index	A slot number that is too large silently lands on the next floor (floor 0, slot 55 with 50 slots per floor gives index 55, which is floor 1, slot 5).	An out-of-range floor or slot throws ArrayIndexOutOfBoundsException, but a padding cell is a valid index although it is not a real slot.	Any out-of-range index throws ArrayIndexOutOfBoundsException, and every valid index is a real slot because there is no padding.
Advantages	One allocation; simple full scan.	Simple to declare and read; slots[floor][slot] notation.	Exact capacity for every floor; same slots[floor][slot] notation; no padding.
Limitations	Only uniform floors; needs index arithmetic; weak protection from invalid indexes.	Wastes memory and needs extra bookkeeping when floors differ.	The rows are separate objects, and the number of floors is fixed once the outer array is created.

Choice: jagged array. The jagged array String[floor][slot] is selected. The stated facilities are uniform (1 × 20, 3 × 50, 5 × 100), but the design must remain correct as the facility grows, and a jagged array handles uniform and unequal floors with the same code and no padding. It also matches the assignment's representation, in which slots[floor][slot] holds a vehicle number or null. The one-dimensional array and the rectangular array cannot represent unequal floors correctly without extra structures.

Code 1: Creating the jagged array and accessing a slot

```
int[] slotsPerFloor = {20, 50, 40};   // illustrative capacities
String[][] slots = new String[slotsPerFloor.length][];
for (int f = 0; f < slots.length; f++) {
    slots[f] = new String[slotsPerFloor[f]];   // null = free
}

slots[1][7] = "TN01AB1111";            // park on floor 1, slot 7
boolean occupied = slots[1][7] != null; // true
int capacityOfFloor1 = slots[1].length; // 50
```

Meaning of the indexes. In slots[floor][slot], the first index is the floor and is counted from 0, so it selects one row. The second index is the slot position within that floor, also counted from 0, so it selects one element of that row. The value null means the slot is free; any other value is the registration number of the parked vehicle. The length of a row, slots[floor].length, is the capacity of that floor, and the total number of slots is the sum of all row lengths.

Preventing invalid indexing. Java checks every array index at run time and throws ArrayIndexOutOfBoundsException for a bad one. The program checks the position first, so that a bad request is rejected with a message instead of an exception:

```
static boolean isValidPosition(String[][] slots, int floor, int slot) {
    return floor >= 0 && floor < slots.length
        && slot >= 0 && slot < slots[floor].length;
}
```

The conditions are joined with && and are evaluated from left to right, so slots[floor].length is evaluated only after the floor has been shown to be valid. The slot is compared with the length of that particular floor, not with a single global limit, which is what makes the check correct for floors of different sizes.

4. Input Handling

Table 4: Scanner compared with command-line arguments

Aspect	Scanner	Command-line arguments (args)
When the data is supplied	While the program runs, one prompt at a time.	Once, when the program is launched, before it starts.
Repeated operations	Natural: entries and exits can be processed one after another in a loop.	Awkward: every value for every operation must be known at launch, or the program must be started again.
Invalid input	The program can explain the error and ask again.	The program can only reject the whole run or ignore the value.
Type of the data	Text read from the keyboard, converted by the program (nextLine, then parse).	Always an array of String values that must be converted by the program.

Aspect	Scanner	Command-line arguments (args)
Suitability	Suited to an interactive parking operator who enters vehicles one at a time.	Suited to a fixed one-shot configuration.

Decision: Scanner. The parking system handles a sequence of entries and exits, each needing several values, and an operator can make typing mistakes. Scanner allows each value to be validated immediately and requested again, whereas command-line arguments would force all data to be supplied in advance with no chance of correction. All input is read with nextLine() and converted explicitly. This avoids the common problem in which nextInt() leaves the end-of-line character in the buffer and a following nextLine() returns an empty string.

Table 5: How each input is received and validated

Input	How it is received	Validation
Vehicle registration number	nextLine(), then trim() and toUpperCase().	Must match [A-Z0-9]{4,12} (assumed rule, lenient enough for values such as TN01A1 and TN01AB1111). Blank text is rejected. That the vehicle is not already parked is checked by the facility (Task 1 rule BR2), not by the input method.
Vehicle type	nextLine(), trim(), toUpperCase(), then VehicleType.valueOf.	valueOf throws IllegalArgumentException for an unknown name; this is caught and the question is asked again.
Floor and slot	Only when an operation names a position (for example releasing a slot). Each is read as text and converted with Integer.parseInt. When a vehicle enters, the system chooses the slot and reports it.	NumberFormatException is caught. The floor must be from 0 to slots.length − 1. The slot must be from 0 to slots[floor].length − 1, so the floor is read first because the permitted slot range depends on it.
Entry time (and exit time)	nextLine(), parsed with LocalDateTime.parse using the pattern uuuu-MM-dd HH:mm.	DateTimeParseException is caught. A strict resolver is used so an impossible date such as 2026-02-30 is rejected instead of being silently adjusted. The exit time must not be before the entry time (Task 1 rule BR6).
Parking duration	Not typed; it is computed from the entry and exit times. If a duration were typed instead, it would be read with the same integer routine.	Must be a whole number of minutes that is not negative.

Code 2: Reading and validating input with Scanner (excerpt)

```
import java.time.LocalDateTime;
import java.time.format.*;
import java.util.Scanner;

static final DateTimeFormatter FMT =
    DateTimeFormatter.ofPattern("uuuu-MM-dd HH:mm")
        .withResolverStyle(ResolverStyle.STRICT);

static String readRegNo(Scanner in) {
    while (true) {
        System.out.print("Registration number: ");
        String reg = in.nextLine().trim().toUpperCase();
        if (reg.matches("[A-Z0-9]{4,12}")) return reg;
        System.out.println("Use 4 to 12 letters or digits.");
    }
}

static VehicleType readType(Scanner in) {
    while (true) {
        System.out.print("Type (TWO_WHEELER, CAR, ELECTRIC_CAR): ");
        try { return VehicleType.valueOf(in.nextLine().trim().toUpperCase()); }
        catch (IllegalArgumentException e) { System.out.println("Unknown type."); }
    }
}

static int readInt(Scanner in, String prompt, int min, int max) {
    while (true) {
        System.out.print(prompt);
        try {
            int v = Integer.parseInt(in.nextLine().trim());
            if (v >= min && v <= max) return v;
        } catch (NumberFormatException e) { /* ask again */ }
        System.out.println("Enter a whole number from " + min + " to " + max);
    }
}

static LocalDateTime readTime(Scanner in, String label) {
    while (true) {
        System.out.print(label + " (yyyy-MM-dd HH:mm): ");
        try { return LocalDateTime.parse(in.nextLine().trim(), FMT); }
        catch (DateTimeParseException e) { System.out.println("Invalid date-time."); }
    }
}

// use: floor first, because the slot range depends on the floor
int floor = readInt(in, "Floor: ", 0, slots.length - 1);
int slot  = readInt(in, "Slot: ", 0, slots[floor].length - 1);
```

5. Type Conversion, Casting and Precision

Where conversion is required. The parking system converts data at five points, and each has a risk that the design removes.

Table 6: Conversion points, risks and the safe approach

Conversion point	Conversion	Risk	Safe approach
Keyboard text to numbers and times	Integer.parseInt and LocalDateTime.parse	Invalid text throws an exception.	Catch the exception and ask again (Section 4).
Entry and exit time to duration	Duration.between(entry, exit).toMinutes() returns a long; the duration is an int	A plain (int) cast silently wraps a value outside the int range and gives a wrong, possibly negative, duration.	Math.toIntExact, which throws ArithmeticException instead of wrapping.
Minutes to billable hours	int minutes to int hours	Integer division truncates and under-charges.	Ceiling division (minutes + 59) / 60.
Hours × rate to fee	int hours is widened to double	None; widening is exact for these values.	No explicit cast is needed.
Fee to whole rupees for display	double to int	A (int) cast truncates: (int) 59.99 is 59.	Keep the fee as double; round with Math.round if whole rupees are ever shown.

Billing rule. Every started hour is charged in full. This is the rule assumed in Task 1, and it agrees with the explicit rule in the assignment's fee code, which states that every started hour is charged in full and asks for the ceiling of minutes / 60 using integer arithmetic. The two do not conflict, so the assignment's rule is followed.

Worked example. A vehicle is parked for 61 minutes at Rs 30 per hour. One hour is complete and a second hour has started, so the billable duration is 2 hours and the correct fee is 2 × 30 = Rs 60.

Table 7: Wrong and correct ways to compute the billable hours for 61 minutes

Expression	Hours	Fee at Rs 30	Verdict
61 / 60	1	Rs 30	Wrong. Both operands are int, so the quotient 1.0166… is truncated to 1 (61 = 60 × 1 + 1; the remainder is discarded). The vehicle is under-charged by Rs 30.

Expression	Hours	Fee at Rs 30	Verdict
(double) (61 / 60)	1.0	Rs 30	Wrong. The cast is applied after the integer division, when the fraction has already been lost (premature conversion).
Math.ceil(61 / 60)	1.0	Rs 30	Wrong. The argument is evaluated first as the int 1, so Math.ceil receives 1.0 and has nothing to round up.
Math.ceil(61 / 60.0)	2.0	Rs 60	Correct. 60.0 is a double, so the division is done in floating point (1.0166…) before Math.ceil.
(61 + 59) / 60	2	Rs 60	Correct. 120 / 60 = 2 using integer arithmetic only.

Code 3: Duration, billable hours and fee

```
int wrongHours = 61 / 60;           // 1   (integer division truncates)
double wrongFee = wrongHours * 30.0; // 30.0, under-charged by 30.0

long minutes = Duration.between(entry, exit).toMinutes();
int durationMinutes = Math.toIntExact(minutes);  // throws if out of int range
int billableHours = (durationMinutes + 59) / 60; // 61 gives (61 + 59) / 60 = 2
double fee = billableHours * ratePerHour;        // 2 * 30.0 = 60.0
```

Assumptions behind (durationMinutes + 59) / 60.

- **Non-negative duration.** The expression equals the ceiling of durationMinutes / 60 only for durationMinutes ≥ 0. Write durationMinutes = 60q + r with 0 ≤ r < 60. If r = 0, (60q + 59) / 60 = q. If r > 0, then 60 ≤ r + 59 ≤ 118, so (60q + r + 59) / 60 = q + 1. For a negative value, integer division truncates towards zero and gives a wrong answer (for −61 the expression gives 0, while the true ceiling of −1.0166… is −1). The program therefore rejects an exit time that is before the entry time (Task 1 rule BR6).
- **Zero minutes.** (0 + 59) / 60 = 0, so a stay of zero minutes has no started hour and costs Rs 0. This is an assumption. If a minimum charge of one hour were required, it would have to be written as a separate rule, for example Math.max(1, billableHours), and it is not assumed here.
- **Overflow.** The addition durationMinutes + 59 overflows an int when durationMinutes is greater than Integer.MAX_VALUE − 59 = 2,147,483,588 (about 4,085 years of minutes). The result wraps to a negative number; for example, 2,147,483,589 minutes would give −35,791,394 billable hours and a negative fee. This cannot happen in a real stay, but the code does not rely on that: the duration is validated to lie between 0 and Integer.MAX_VALUE − 59, or the overflow-free form durationMinutes / 60 + (durationMinutes % 60 == 0 ? 0 : 1) is used, which is correct for every non-negative int.

Floating-point alternative. The ceiling can also be computed as (int) Math.ceil(durationMinutes / 60.0). The divisor must be written 60.0 (or the numerator cast to double before the division) so that the division is performed in floating point;

Math.ceil(durationMinutes / 60) divides as ints first and is wrong, as Table 7 shows. For an int durationMinutes the floating-point form gives the correct result, because a multiple of 60 divides exactly and every other value leaves a fraction of at least 1/60, which is far larger than any rounding error. The integer form is preferred because it needs no conversion to double and no reasoning about rounding.

How incorrect casts and premature conversion lose precision.

- **Cast after the division.** (double) (minutes / 60) converts a value from which the fraction was already discarded. The cast has to come before the division, as in (double) minutes / 60, or the arithmetic has to stay in integers.
- **Narrowing a long.** (int) minutes keeps only the low 32 bits. For example, (int) 4,294,967,356L is 60, so a duration of 4,294,967,356 minutes would be billed as one hour. Math.toIntExact throws an exception in this case instead.
- **Narrowing a rate.** Casting a rate to int discards its fraction (an hourly rate of 12.5 would become 12). The design keeps rates as double and never casts them.
- **Truncating a fee.** (int) 59.99 gives 59, which under-states the amount by Rs 0.99, while Math.round(59.99) gives 60.

How the design avoids these errors. Duration is an int number of minutes, billable hours are calculated with integer ceiling division after the duration has been checked to be non-negative and within range, the fee is computed as int × double with the widening done automatically, and the fee is never cast to int.

6. Alternative Concept and Complexity Analysis

The question. Instead of the jagged array String[floor][slot], the slots could be stored in a single one-dimensional array with index = floor × slotsPerFloor + slot, or in an ArrayList<String[]>. The analysis uses these symbols: F is the number of floors, capacity[f] is the number of slots on floor f, and N is the total number of slots (the sum of all capacity[f]). Every floor is assumed to have at least one slot, so $F \leq N$.

What changes when floors have different numbers of slots. The simple one-dimensional formula assumes the same number of slots on every floor, so it is correct only for uniform floors. For unequal floors it needs cumulative offsets (an int array in which offset[f] is the number of slots on all floors before floor f) or a mapping table, and the index becomes offset[floor] + slot. The slot must also be checked against capacity[floor] before the index is computed, because a slot number that is too large would otherwise land on the next floor. Whenever a found index has to be turned back into a floor and slot, the uniform case needs one division and one remainder, while the offset case needs a search of the offset table. An ArrayList<String[]> can hold rows of different lengths without any extra table, but every access needs two checks: the floor against rows.size() and the slot against rows.get(floor).length. The jagged array needs no change at all; each row simply has its own length.

Code 4: One-dimensional array (uniform floors, and unequal floors with offsets)

```
// uniform floors: every floor has slotsPerFloor slots
String[] flat = new String[floors * slotsPerFloor];
flat[floor * slotsPerFloor + slot] = regNo;

// unequal floors: cumulative offsets
int[] capacity = {20, 50, 40};
int[] offset = new int[capacity.length + 1];
for (int f = 0; f < capacity.length; f++) {
    offset[f + 1] = offset[f] + capacity[f];     // offsets 0, 20, 70, 110
}
String[] flatUnequal = new String[offset[capacity.length]];
if (slot >= 0 && slot < capacity[floor]) {     // check before indexing
    flatUnequal[offset[floor] + slot] = regNo;
}
```

Code 5: ArrayList<String[]> (requires import java.util.ArrayList)

```
ArrayList<String[]> rows = new ArrayList<>();
int[] capacity = {20, 50, 40};
for (int c : capacity) {
    rows.add(new String[c]);              // one row per floor
}

if (floor >= 0 && floor < rows.size()
    && slot >= 0 && slot < rows.get(floor).length) {
    rows.get(floor)[slot] = regNo;
}
rows.add(new String[60]);              // a new floor can be added later
```

Table 8: Comparison of the jagged array with the two alternatives

Aspect	Jagged array String[][] (chosen)	One-dimensional String[]	ArrayList<String[]>
Representation	Outer array of F rows; row f has capacity[f] elements.	One array of N (or F × slotsPerFloor) elements; index = floor × slotsPerFloor + slot.	List of F elements; each element is a String[] for one floor.
Unequal floor capacities	Handled directly; each row has its own length.	Formula is valid only for uniform floors; needs an offset table (or padding to the largest floor).	Handled directly; each row has its own length.
Access by floor and slot	Check against slots.length and slots[floor].length, then slots[floor][slot].	Check slot against capacity[floor], then flat[offset[floor] + slot] (or the uniform formula).	Check against rows.size() and rows.get(floor).length, then rows.get(floor)[slot].

Aspect	Jagged array String[][] (chosen)	One-dimensional String[]	ArrayList<String[]>
Locate a free slot	Scan floor by floor and slot by slot until a null element is found.	One loop over indexes until a null element is found; then convert the index to floor and slot.	Scan the rows in list order and the slots in each row until a null element is found.
Time complexity	Direct access O(1). Free-slot search O(N) worst case, O(1) best case.	Direct access O(1). Free-slot search O(N) worst case; converting the found index costs O(1) (uniform) or O(log F) with binary search of the offsets, so the total stays O(N).	Direct access O(1). Free-slot search O(N) worst case, O(1) best case.
Space complexity	O(N + F): N element references, F row arrays and the outer array.	O(N) for uniform floors; O(N + F) with an offset table; O(F × largest capacity) if padded.	O(N + F): the same rows plus the list object and its backing array (capacity at least F).
Advantages	Exact capacity per floor; same slots[floor][slot] notation; no extra table.	A single allocation; very simple scan.	Floors can be added or removed at run time without rebuilding the structure.
Limitations	The number of floors is fixed once the outer array exists; adding a floor means building a new outer array and copying F references.	Breaks for unequal floors unless offsets are kept; a wrong slot number can silently reach another floor; changing capacities means rebuilding the array and the offsets.	Two validations per access; get throws IndexOutOfBoundsException for a bad floor while the row throws ArrayIndexOutOfBoundsException for a bad slot.

Direct access compared with searching. When the floor and slot are already known (for example when a slot is released), the position is reached by a fixed number of array accesses and, for the one-dimensional array, one arithmetic step and one offset lookup. This is O(1) in all three representations, independent of F and N, so none is asymptotically faster. Their constant factors differ slightly (one array access for the flat array, two for the others), but no measurement was made, and the difference is not significant for facilities of at most 500 slots.

When no position is known, a free slot must be found by scanning. In the worst case (a full facility) all N elements are examined, and in the best case the first element is free, so the search is O(N) in the worst case and O(1) in the best case for all three representations. Each examination is a null test, which costs O(1). The nested loops of the jagged array and the ArrayList add a small cost of O(F) for moving between rows, and O(N + F) = O(N) because $F \leq N$. The three representations therefore have the same asymptotic time for both operations.

For space, all three grow linearly with the number of slots: O(N + F) for the jagged array and the ArrayList, and O(N) or O(N + F) for the one-dimensional array. The difference is in constant

factors (row headers, the list object and its backing array, the offset table) and in the padding that a uniform array would need for unequal floors, O(F × largest capacity) instead of O(N).

Assumptions of the analysis. N is the total number of slots, F ≥ 1 is the number of floors with F ≤ N, and every floor has at least one slot. The search scans in floor-major order with no auxiliary free-slot list. Array element access, ArrayList.get and a null test are O(1). The space of the registration-number Strings is the same for every representation (one String per parked vehicle) and is not counted. Big-O ignores constant factors. This task gives an analytical comparison only; no timing or memory measurement was taken.

7. Design Decision

Java data types. Slot, floor, duration and billable hours are int; the registration number is a String; the vehicle type is an enum; entry and exit times are LocalDateTime; rates and fees are double; the ticket number is a long; occupancy is a boolean derived from the slot content. Each type holds its data without loss and is checked by the compiler or at parsing time, so mistakes are caught early (correctness). Types that match the assignment's own code keep the later classes consistent (maintainability), and moving the fee to BigDecimal later would be limited to the fee and rate fields (extension).

Floor-and-slot structure. The jagged array String[floor][slot] is used. It is correct for uniform and unequal floors, has no padding slots that could be allocated by mistake, and allows each position to be validated against the true capacity of its floor (correctness). It is simple and matches the assignment's notation (maintainability), and it supports floors with different capacities without changing any code (extension).

Input method. Scanner with nextLine() and explicit parsing is used. Each value is validated at once and requested again if wrong, so invalid data never reaches the facility (correctness). One reading style and a small set of helper methods keep the input code uniform (maintainability), and a new input field needs only one more helper (extension).

Duration conversion and fee calculation. The duration is an int number of minutes, the billable hours are (durationMinutes + 59) / 60 for a validated non-negative duration, and the fee is billable hours × hourly rate in double. This implements the rule that every started hour is charged in full without the truncation error of 61 / 60, uses no floating-point rounding, and guards the long-to-int narrowing and the overflow limit (correctness). The rule is stated in one place (maintainability), and a different rounding rule or a BigDecimal fee could replace it without changing the callers (extension).

8. Result

The data representation was decided as follows. The registration number is a String, the vehicle type an enum, the entry and exit times LocalDateTime values, the duration an int number of minutes, the fee and rate double values, the ticket number a long, and slot occupancy a boolean derived from slots[floor][slot] != null. Floors and slots are held in a jagged array String[floor] [slot], and input is read with Scanner.

The analysis established that integer division would bill 61 minutes as one hour (Rs 30) instead of two (Rs 60), and that ceiling division, (61 + 59) / 60 = 2, avoids this error for non-negative durations. It also established that a one-dimensional array with the formula floor × slotsPerFloor + slot is valid only for uniform floors, that an ArrayList<String[]> supports unequal floors but needs two index checks, and that direct access is O(1) and a free-slot search is O(N) in the worst case for all three representations, with space growing linearly with the

number of slots. The code snippets in this report are illustrative excerpts and not a standalone program, and no measured results are reported.